2024 中山大学程序设计校赛 网络赛题解

ACMM 协会

A

根据题意，在长度固定时，对应的最终字符串也是固定的。此时只需要构造出这个字符串并进行匹配即可。

- 方法 1：新建一个字符串变量，不断在后面追加 `SYSU` 直到长度等于 n；
- 方法 2：只需要知道最终的字符串在 $4k+1$ 位和 $4k+3$ 位上为 `S`，$4k+2$ 位上为 `Y`，$4k$ 位上为 `U` 即可。

时间复杂度 $O(n)$。

B

我们贪心考虑，每次都让 WAduck 和一只实力比他低的鸭子打。

由于 WAduck 的可爱值是 k，一共有可爱值为 $0, 1, ..., k-1$ 共 k 只鸭子可以输给 WAduck 。

对于第 i 场比赛，WAduck 需要打败一只实力比他弱的鸭子，同时，剩下的实力比 WAduck 弱的鸭子里，经过这一轮比赛后个数一定至少减半（如果是奇数个则先减一再减半）。

也就是说，假设第 i 轮后剩余的实力比 WAduck 弱的鸭子至多有 k_i 只，那么经过一轮比赛后，在最好情况下：

$$k_{i+1} = \left\lfloor \frac{k_i - 1}{2} \right\rfloor$$

当某一轮所有实力比 WAduck 低的鸭子都已经被淘汰，WAduck 必定在这一轮结束。故只需要计算出 k_i 数组并比对即可。复杂度 $O(\log k)$。

C

我们思考一下每个数字都出现两次代表了什么。

我们可以将其理解为平面上有 n 个点，有 n 条无向边，每条边连接的是每个硬币两面的数字，并且每个点度数都为 2。我们需要给每个无向边规定一个方向（其中一个方向代表翻正面，另一个代表翻反面），使得每个点的入度都为 1。

思考这个图的构造，很明显是若干个互不联通的环。对于每一个环，很明显有 2 种边的分配方式。那么如果一共有 k 个环，那么一共有 2^k 种分配方式。

接下来就是求出有多少个互相不连通的环。我们可以使用并查集，或者使用 dfs 判断连通性。复杂度 $O(n)$ 或 $O(n \log n)$。

D

本题考察容斥的应用和组合数的应用。

对于 $N = K$ 的情况，该问题为经典的错排问题，这个问题可以利用 dp 或者容斥原理进行计算。本题中利用 dp 进行计算会非常复杂，因此我们可以尝试使用容斥原理计算答案。

我们设 f_i 为所有的方案中，钦定 i 个位置正确的方案数。我们首先需要选择 i 个位置，这些位置的答案为正确的，方案为 $\binom{K}{i}$。 然后我们需要选择剩下的 $N-i$ 个答案中的 $K-i$ 个，作为剩下的位置填入。方案为 $\binom{N-i}{K-i} \times (K-i)!$。因此：

$$f_i = \binom{K}{i} \times \binom{N-i}{K-i} \times (K-i)!$$

接下来我们思考容斥系数的问题。

考虑一种方案，实际填对的题目数为 j，那么对于每个 $i(i \le j)$，j 在 f_i 中被计算了 $\binom{j}{i}$ 次。

由于 $\sum_{i=0}^{j} (-1)^i \binom{j}{i} = 0$，我们只需要将容斥系数设为 $(-1)^i$ 即可计算答案。

最终答案为

$$\sum_{i=0}^{K} (-1)^i \binom{K}{i} \binom{N-i}{K-i} (K-i)!$$

预处理阶乘和阶乘逆元计算即可。复杂度$O(N \log N)$，也可以线性求逆元做到$O(N)$。

E

我们考虑如果在某一回合没有打出伤害，那么这一回合打的卡就算全移除对结果也不会产生影响，也就是每一回合的伤害 s_i 一定有 $s_i > D$。那么假设如果总共打了 m 回合，那么伤害一定是 $\sum_{i=1}^{m} s_i - D \times m$，而此时我们只需要最大化 $\sum_{i=1}^{m} s_i$ 就好了，也就是最大化每一回合伤害的总和，那么直接选出伤害最高的 $m \times k$ 张卡就好。

考虑对于每个 k，m 不会超过 $\frac{n}{k} + 1$ ，并且 $\sum_{k=1}^{n} \frac{n}{k}$ 是 $n \log n$ 级别的。所以我们先把卡的伤害从大到小排序，然后做一个前缀和，对于每个 k 暴力枚举 m 用前缀和计算答案就好了，时间复杂度 $O(n \log n)$。

F

因为只有三个 Badeline，所以我们只需要考虑 Madeline 最近 3 次的移动情况。首先除了第一次就不能回头走了，其次最近四次的移动方向不能完全不同（也就是不能最近四次移动方向上下左右全都有）。

那么我们可以设 $f_{n,i,j,k}$ 表示移动了 n 次，最近三次的移动方向分别为 i, j, k 的情况下有多少种方案。这样总共有 $n \times 4^3$ 种状态。

当然还有更好写的做法，我们可以设 $f_{n,x,y}$ 表示移动了 n 次，x 为 $0/1/2$ ，分别表示最后一次移动与倒数第二次移动方向相同/顺时针转了 $90°$/逆时针转了 $90°$。y 同样为 $0/1/2$，表示的是倒数第二次移动与倒数第三次移动的方向区别。那么如果你不是连续三次顺时针转或者连续三次逆时针转就可以。这样状态数就压到了 $n \times 3^2$。

两种做法都能通过，时间复杂度 $O(n)$。

G

如果有环，那么就直接找到一个环定向，然后其他的点都指向这个环就行了。

然后考虑是树的情况怎么做，我们随意找一个根开始出发，对于每个位置记录一个 f_i 表示 i 号点的子树最多能延伸一条向下长度为 f_i 的路径，如果 f_i 是

负数则表示不能往下延伸，且要求至少向上延伸一条长度为 f_i 的路径。这样我们去合并相邻子树时找到一个能往下的最深的，看下其他往上的能不能都塞到那条里去就行了。因为一个子树能向下证明它自身就能够合法，所以能向下就向下一定最优。

时间复杂度 $O(n+m)$。